

Kiro Spec Handbook

Architecture & Specification Reference

just-ak

Build date: 2026-08-06T21:52:07+01:00

Source revision: 6d0ac9a

Generated automatically from Kiro specifications. Do not edit by hand.

Contents

1	Document Information	2
2	Revision History	3
3	Architecture Overview	4
4	Specification Index	5
5	Requirements Index	6
6	Tasks Index	7
7	Diagram Index	8
8	Steering Documents	9
8.0.1	Diagramming	9
8.0.2	Product	9
8.0.3	Project Structure	10
8.0.4	Technology	11
9	Specifications	12
9.1	Handbook Generation — SPEC-HANDBOOK-GENERATION	13
9.1.1	Requirements	13
9.1.2	Design	14
9.1.3	Tasks	15
9.1.4	Diagrams	17
9.2	PDF Chunking — SPEC-PDF-CHUNKING	18
9.2.1	Requirements	18
9.2.2	Design	19
9.2.3	Tasks	19
9.2.4	Diagrams	21
9.3	Tokenizer Summary — SPEC-TOKENIZER-SUMMARY	22
9.3.1	Requirements	22
9.3.2	Design	22
9.3.3	Tasks	23
10	Architecture Decisions	25
11	Traceability Matrix	26
12	Git Change History	27
13	Appendices	28
13.1	Appendix A — Reference Files	28
13.2	Appendix B — Tokenizer Summary	28
13.2.1	Specifications	28
13.2.2	Steering Documents	28

1 Document Information

Field	Value
Title	Kiro Spec Handbook
Organisation	just-ak
Build date	2026-08-06T21:52:07+01:00
Source revision	6d0ac9a
Specifications	3
Requirements	12
Tasks	44
Diagrams	2
Source of truth	.kiro/specs

This handbook is generated automatically from the Kiro specifications. Do not edit it by hand — regenerate with `yarn handbook build`. The specs remain the single source of truth.

2 Revision History

Revision	Date	Notes
v0.1.0	2026-08-06	Release
6d0ac9a	2026-08-06T21:52:07+01:00	Handbook build

3 Architecture Overview

No dedicated `architecture` spec was found in the source tree.

Status	Specs
implemented	3

4 Specification Index

Spec ID	Title	Ver	Status	Reqs	Tasks	Figs
SPEC-HANDBOOK-GENERATION	Handbook Generation	1.0	implemented	5	18	1
SPEC-PDF-CHUNKING	PDF Chunking	1.0	implemented	4	14	1
SPEC-TOKENIZER-SUMMARY	Tokenizer Summary	1.0	implemented	3	12	0

5 Requirements Index

Requirement ID	Requirement
SPEC-HANDBOOK-GENERATION:R1	Req 1: Spec discovery and parsing
SPEC-HANDBOOK-GENERATION:R2	Req 2: Stable identifiers
SPEC-HANDBOOK-GENERATION:R3	Req 3: Indexes and cross-references
SPEC-HANDBOOK-GENERATION:R4	Req 4: Deterministic assembly
SPEC-HANDBOOK-GENERATION:R5	Req 5: Graceful degradation
SPEC-PDF-CHUNKING:R1	Req 1: Page-based splitting into valid PDFs
SPEC-PDF-CHUNKING:R2	Req 2: Configurable and auto-detected size
SPEC-PDF-CHUNKING:R3	Req 3: Oversized single pages
SPEC-PDF-CHUNKING:R4	Req 4: Predictable naming and dry-run
SPEC-TOKENIZER-SUMMARY:R1	Req 1: Per-model token estimation
SPEC-TOKENIZER-SUMMARY:R2	Req 2: Configurable models
SPEC-TOKENIZER-SUMMARY:R3	Req 3: Summary appendix

6 Tasks Index

Task ID	Done	Task
SPEC-HANDBOOK-GENERATION:T1	☑	Configuration loading
SPEC-HANDBOOK-GENERATION:T2	☑	Spec discovery and parsing
SPEC-HANDBOOK-GENERATION:T2.1	☑	Discover spec directories and classify documents
SPEC-HANDBOOK-GENERATION:T2.2	☑	Sort specs by stable id after discovery
SPEC-HANDBOOK-GENERATION:T2.3	☑	Compute deterministic content hashes
SPEC-HANDBOOK-GENERATION:T3	☑	Stable identifiers
SPEC-HANDBOOK-GENERATION:T4	☑	Indexes and cross-references
SPEC-HANDBOOK-GENERATION:T4.1	☑	Build specification, requirements, tasks, and diagram indexes
SPEC-HANDBOOK-GENERATION:T4.2	☑	Linkify requirement references from tasks
SPEC-HANDBOOK-GENERATION:T4.3	☑	Emit the traceability matrix
SPEC-HANDBOOK-GENERATION:T4.4	☑	Assign handbook-wide figure numbers
SPEC-HANDBOOK-GENERATION:T5	☑	Deterministic assembly
SPEC-HANDBOOK-GENERATION:T6	☑	Rendering and graceful degradation
SPEC-HANDBOOK-GENERATION:T6.1	☑	Assemble Markdown (pure) and write it to the output directory
SPEC-HANDBOOK-GENERATION:T6.2	☑	Detect Pandoc/LaTeX and render the PDF when available
SPEC-HANDBOOK-GENERATION:T6.3	☑	Convert and embed SVG diagrams; reference them when no converter exists
SPEC-HANDBOOK-GENERATION:T6.4	☑	Render mermaid via mmdc; fall back to fenced code when absent
SPEC-HANDBOOK-GENERATION:T7	☑	Lock file and delta detection
SPEC-PDF-CHUNKING:T1	☑	Load and measure the input PDF
SPEC-PDF-CHUNKING:T2	☑	Single-chunk fast path
SPEC-PDF-CHUNKING:T3	☑	Greedy page packing
SPEC-PDF-CHUNKING:T3.1	☑	Measure real serialised size of each tentative chunk
SPEC-PDF-CHUNKING:T3.2	☑	Close and start groups on overshoot; preserve page order
SPEC-PDF-CHUNKING:T3.3	☑	Emit oversized single pages with a warning
SPEC-PDF-CHUNKING:T4	☑	Size selection
SPEC-PDF-CHUNKING:T4.1	☑	Validate explicit sizes (1–4 MB)
SPEC-PDF-CHUNKING:T4.2	☑	Auto-size heuristic within the 4 MB limit
SPEC-PDF-CHUNKING:T5	☑	Output, naming, and reporting
SPEC-PDF-CHUNKING:T5.1	☑	Name chunks <i>chunkof.pdf</i>
SPEC-PDF-CHUNKING:T5.2	☑	Dry-run preview without writing files
SPEC-PDF-CHUNKING:T5.3	☑	Summary of totals and per-chunk stats
SPEC-PDF-CHUNKING:T6	☑	CLI wiring
SPEC-TOKENIZER-SUMMARY:T1	☑	Token counting primitives
SPEC-TOKENIZER-SUMMARY:T2	☑	Model profiles and resolution
SPEC-TOKENIZER-SUMMARY:T2.1	☑	Define built-in Claude and GPT profiles
SPEC-TOKENIZER-SUMMARY:T2.2	☑	Apply per-model chars_per_token overrides
SPEC-TOKENIZER-SUMMARY:T2.3	☑	Fall back to a generic profile for unknown keys
SPEC-TOKENIZER-SUMMARY:T2.4	☑	Default to a single model when none configured
SPEC-TOKENIZER-SUMMARY:T3	☑	Summary appendix rendering
SPEC-TOKENIZER-SUMMARY:T3.1	☑	Build per-spec and per-steering rows
SPEC-TOKENIZER-SUMMARY:T3.2	☑	Sort largest-first and add a totals row
SPEC-TOKENIZER-SUMMARY:T3.3	☑	Fold the summary into Appendices as Appendix B
SPEC-TOKENIZER-SUMMARY:T3.4	☑	Emit nothing when disabled
SPEC-TOKENIZER-SUMMARY:T4	☑	Configuration

7 Diagram Index

Figure	Diagram ID	Caption	Spec
Figure 1	FIG-HANDBOOK- GENERATION-1	Pipeline	SPEC-HANDBOOK- GENERATION
Figure 2	FIG-PDF-CHUNKING-1	Chunking Flow	SPEC-PDF-CHUNKING

8 Steering Documents

Project-wide rules and guidance from `.kiro/steering`, reproduced in full.

8.0.1 Diagramming

Rule: architecture diagrams are always SVG All architecture diagrams committed to a spec **must be authored as SVG** and placed in the spec's `diagrams/` folder (or at the spec root). This is a hard requirement, not a preference.

Why SVG

- **Vector, not raster.** SVGs stay crisp at any print size and on high-DPI tablets — essential for a handbook meant to be printed and annotated.
- **Diff-able and reviewable.** SVG is text (XML), so diagram changes show up in version control and code review like any other source.
- **First-class in the pipeline.** The scanner discovers `*.svg` automatically, assigns each a stable `FIG-*` id and a handbook-wide figure number, and the renderer converts SVG → PDF (via `rsvg-convert/inkscape/cairosvg`) to embed them as full landscape figures.
- **Deterministic.** Text-based sources hash cleanly and reproduce identically across builds.

Authoring guidelines

- Name files descriptively; the file name becomes the figure caption (`pipeline.svg` → “Pipeline”, `chunking-flow.svg` → “Chunking Flow”).
- Prefer a landscape aspect ratio (e.g. `viewBox="0 0 1200 700"`); diagrams are placed on their own landscape page.
- Set an explicit `viewBox` and use readable font sizes (≥ 14 px in diagram units).
- Keep text as real `<text>` elements (not outlined paths) so it stays searchable and legible.
- Use a restrained palette and label every node and edge.

Mermaid is the exception, not the rule Inline `mermaid` code blocks in requirements/design/tasks are supported and rendered to SVG at build time (via `mmdc`) for convenience. But the **canonical architecture diagrams** — the ones that define the system — should be committed as hand-maintained SVG files so they are stable, reviewable, and never depend on an optional renderer being installed.

8.0.2 Product

`kiro-spec-handbook` converts AWS Kiro specifications into a professionally formatted, indexed PDF handbook. The handbook is meant to be printed or read on a tablet (reMarkable, iPad), annotated by hand, and then fed back into the project so the notes can be processed into spec changes.

Who it is for Teams and individuals who build software with Kiro specs and want a durable, reviewable artifact of their specifications — one that works away from the screen and supports collaborative, handwritten review.

Core value

- A single, indexed document assembled from many spec files.
- Stable identifiers for specs, requirements, tasks, and diagrams that never depend on file-names or page numbers.
- Deterministic builds suitable for CI and version control.
- PDF chunking so large handbooks fit within Kiro's 4 MB attachment limit.
- A tokenizer summary that surfaces bloat in specs and steering docs.

Guiding principles

- **Source of truth is the specs.** The handbook is always generated; never edit the output by hand.
- **Determinism first.** Same inputs produce the same bytes. Sort by stable id, hash content, and derive the build date from git or `SOURCE_DATE_EPOCH`.
- **Degrade gracefully.** If Pandoc, LaTeX, an SVG converter, or `mmdc` is missing, still produce useful Markdown rather than failing hard.
- **The tool works on a fresh checkout.** Sensible defaults apply when config or optional tooling is absent.

8.0.3 Project Structure

Layout

```
kiro-spec-handbook/  
  src/                # TypeScript source (one module per concern)  
  test/              # vitest suites + shared fixtures  
  templates/         # LaTeX template (handbook.latex)  
  examples/          # Sample PDFs used for chunking demos  
  .kiro/  
    specs/           # This project's own specifications (dogfooding)  
    steering/        # Project-wide rules (this folder)  
    handbook.yml     # Handbook build configuration  
  dist/              # Compiled output (published, git-ignored)
```

Module map (`src/`) The pipeline modules, in the order data flows through them:

- `config.ts` — load + normalise `handbook.yml`; locate repo/tool roots.
- `scanner.ts` — discover specs, parse documents, compute content hashes.
- `markdown.ts` — parse requirement/task headings; derive captions.
- `metadata.ts` — front matter parsing and stable id derivation.
- `diagrams.ts` — SVG discovery and handbook-wide figure numbering.
- `indexer.ts` — spec/requirement/task/diagram indexes + traceability matrix.
- `references.ts` — cross-reference anchors and hyperlinks.
- `sections.ts` — document info, architecture, steering, appendices, tokenizer.
- `tokenizer.ts` — deterministic per-model token estimation.
- `changelog.ts` + `git.ts` — revision history and change log from git.
- `renderer.ts` — assemble Markdown and drive Pandoc/LaTeX to PDF.
- `lock.ts` — `handbook.lock.json` for delta detection between builds.
- `pdf-chunker.ts` + `pdf-chunker-cli.ts` — split PDFs for Kiro upload.
- `index.ts` — CLI entry (handbook command).

Spec authoring conventions Each spec is a directory under `.kiro/specs/<slug>/` containing:

- `requirements.md` — `### Requirement N: Title` headings (EARS acceptance criteria beneath). This file's front matter/H1 is the canonical spec metadata.
- `design.md` — the technical design.
- `tasks.md` — `- [x] N. Title` checkboxes, each linking `_Requirements: x.y_`.
- `diagrams/*.svg` — architecture diagrams (SVG only — see `diagramming.md`).
- `references/**` — supporting files listed in the appendix.

Optional front matter (recognised only at the top of a file):

```
---  
spec_id: SPEC-EXAMPLE-001 # authoritative id; otherwise derived from the slug
```

```
title: Example Feature
version: "1.0" # quote versions so YAML keeps "1.0"
status: draft
---
```

Identifiers are stable: spec ids derive from the directory name, and requirement/task/diagram ids derive from the spec id plus their in-document number (SPEC-EXAMPLE-001:R3, :T4.1, FIG-EXAMPLE-001-2).

8.0.4 Technology

Stack

- **Language:** TypeScript (ES2022, NodeNext modules), compiled with `tsc`.
- **Runtime:** Node.js ≥ 20 . Distributed as ESM (`"type": "module"`).
- **CLI:** `commander` exposes two binaries — `handbook` and `pdf-chunker`.
- **Testing:** `vitest`. Run once with `vitest run`.

Key libraries

- `globby` — spec/file discovery.
- `gray-matter` + `js-yaml` — front matter and config parsing.
- `pdf-lib` — reading and splitting PDFs for chunking.
- `execa` — invoking external tools (Pandoc, SVG converters, `mmdc`).
- `chalk` — CLI logging.

External tools (optional, degrade gracefully)

- **Pandoc + XeLaTeX** — render assembled Markdown to PDF. Absent $\Rightarrow$ emit Markdown.
- **rsvg-convert / inkscape / cairosvg** — convert SVG $\Rightarrow$ PDF for embedding.
- **@mermaid-js/mermaid-cli** (`mmdc`) — render mermaid blocks to SVG.

The renderer discovers TinyTeX/MacTeX/TeX Live and Homebrew locations even when they are not on `PATH`, so builds work right after a TinyTeX install.

Commands

```
npm run build      # tsc -> dist/
npm run test       # vitest run
npm run handbook   # run the CLI via tsx (e.g. npm run handbook -- build)
npm run chunk      # run the pdf-chunker CLI via tsx
```

CLI subcommands: `handbook build` | `validate` | `index` | `changes` | `chunk`.

Release Automated with semantic-release on push to `main`. Commits follow Conventional Commits (`feat:`, `fix:`, `perf:`, `refactor:`, breaking with `!`). Version, changelog, git tag, and GitHub Packages publish are all handled by CI.

Conventions

- Prefer pure, testable functions; keep IO at the edges (e.g. `assembleMarkdown` is pure so it can be unit-tested).
- Everything user-visible must be deterministic across platforms — normalise path separators to POSIX and line endings to `\n`.
- Never fail the build for missing optional tooling; warn and continue.

9 Specifications

9.1 Handbook Generation — SPEC-HANDBOOK-GENERATION

Version: 1.0 · **Status:** implemented · **Slug:** handbook-generation

9.1.1 Requirements

Introduction The handbook generation pipeline is the core of `kiro-spec-handbook`. It discovers Kiro specifications on disk, parses them into a stable in-memory model, builds cross-referenced indexes, and assembles a single Markdown document that is rendered to a professionally formatted PDF via Pandoc and LaTeX. The pipeline is deterministic and degrades gracefully when optional tooling is absent.

Requirements

Requirement 1: Spec discovery and parsing **User Story:** As a spec author, I want the tool to automatically discover and parse every specification in my project, so that I do not have to register files manually.

Acceptance Criteria

1. WHEN the tool runs THEN the system SHALL treat each immediate subdirectory of the configured specs root as one specification.
2. WHEN a spec directory contains `requirements.md`, `design.md`, or `tasks.md` THEN the system SHALL classify and parse each document by its kind.
3. WHEN a spec contains `references/**` files THEN the system SHALL record them as reference files without treating them as primary documents.
4. WHEN a directory contains no markdown and no SVG files THEN the system SHALL skip it.
5. WHEN discovery completes THEN the system SHALL return specs sorted by their stable id.

Requirement 2: Stable identifiers **User Story:** As a reviewer, I want identifiers that never change between builds, so that references remain valid over time.

Acceptance Criteria

1. WHEN a spec declares `spec_id` in front matter THEN the system SHALL use that value as the authoritative id.
2. WHEN no `spec_id` is declared THEN the system SHALL derive the id from the directory slug as `SPEC-<TOKEN>`.
3. WHEN assigning requirement, task, and diagram ids THEN the system SHALL derive them from the spec id plus the in-document number.
4. WHEN filenames, headings, or page numbers change THEN the system SHALL NOT change any derived identifier.

Requirement 3: Indexes and cross-references **User Story:** As a reader, I want indexes and working cross-references, so that I can navigate a large handbook quickly.

Acceptance Criteria

1. WHEN indexes are enabled THEN the system SHALL emit specification, requirements, tasks, and diagram indexes.
2. WHEN a task references a requirement THEN the system SHALL render a hyperlink to that requirement's anchor.
3. WHEN traceability is enabled THEN the system SHALL emit a traceability matrix linking requirements to the tasks that implement them.
4. WHEN diagrams are present THEN the system SHALL assign sequential, handbook-wide figure numbers in document order.

Pipeline

Figure 1: Pipeline

Requirement 4: Deterministic assembly **User Story:** As a CI maintainer, I want identical inputs to produce identical output, so that builds are reproducible.

Acceptance Criteria

1. WHEN assembling the document THEN the system SHALL order sections deterministically and separate top-level parts with page breaks.
2. WHEN computing the build date THEN the system SHALL use `config.build_date`, then `SOURCE_DATE_EPOCH`, then the last git commit date, in that order.
3. WHEN hashing a spec THEN the system SHALL produce a stable content hash over its source files, independent of platform path separators.

Requirement 5: Graceful degradation **User Story:** As a first-time user, I want the tool to work on a fresh checkout, so that missing optional tooling does not block me.

Acceptance Criteria

1. WHEN Pandoc is not installed THEN the system SHALL still write the assembled Markdown and report that the PDF step was skipped.
2. WHEN no SVG converter is available THEN the system SHALL list diagrams by reference rather than failing.
3. WHEN `mmdc` is not installed THEN the system SHALL render mermaid blocks as fenced code rather than failing.
4. WHEN the configuration file is absent THEN the system SHALL apply built-in defaults.

9.1.2 Design

Overview Handbook generation is a linear pipeline of small, mostly-pure modules. Data flows from spec files on disk into a typed in-memory model, then into assembled Markdown, and finally into a PDF via Pandoc/XeLaTeX. IO is kept at the edges so the assembly logic can be unit-tested in isolation.

Architecture The pipeline stages and their supporting modules are shown below.

Stages:

1. **Config** (`config.ts`) loads and normalises `handbook.yml`, locating the repository root (nearest ancestor containing `.kiro`) and the tool root (which ships the LaTeX template).
2. **Scan** (`scanner.ts`) discovers spec directories, reads each document, classifies it, and computes a deterministic SHA-256 content hash across all source files.
3. **Parse** (`markdown.ts`, `metadata.ts`, `diagrams.ts`) extracts requirements, tasks, front matter, stable ids, and SVG diagram references.
4. **Index** (`indexer.ts`, `references.ts`) builds the indexes, assigns figure numbers, and wires cross-reference anchors and hyperlinks.
5. **Assemble** (`renderer.ts` & `assembleMarkdown`) concatenates every section in a fixed order, separated by LaTeX page breaks. This function is pure.
6. **Render** (`renderer.ts` & `render`) prepares diagram assets, resolves steering file includes, writes the Markdown, and invokes Pandoc when present.
7. **Lock** (`lock.ts`) writes `handbook.lock.json` for delta detection on the next build.

Components and Interfaces

- `scanSpecs(config): Promise<Spec[]>` — discovery + parsing, returns specs sorted by id.
- `assembleMarkdown(handbook, config, parts, images, mermaidImages): string` — pure assembly of the final Markdown.
- `render(handbook, config, parts): Promise<RenderResult>` — full pipeline with IO and Pandoc invocation.

The `HandbookParts` interface carries pre-rendered Markdown fragments (document info, indexes, steering, traceability, appendices) injected into the assembler in order.

Data Model The central types are `Spec` (id, slug, title, version, status, documents, requirements, tasks, diagrams, references, contentHash) and `Handbook` (specs, buildDate, revision). See `src/types.ts`.

Error Handling Missing optional tooling never fails the build: the renderer detects Pandoc, an SVG converter, and `mmdc`, and falls back to Markdown, referenced diagrams, or fenced mermaid code respectively. Validation errors (broken references, bad metadata) are surfaced by the `validate` command and can block build.

Testing Strategy `assembleMarkdown`, id derivation, markdown parsing, indexing, references, and lock/delta logic are unit-tested. An integration test drives `scanSpecs` over a temporary fixture spec tree.

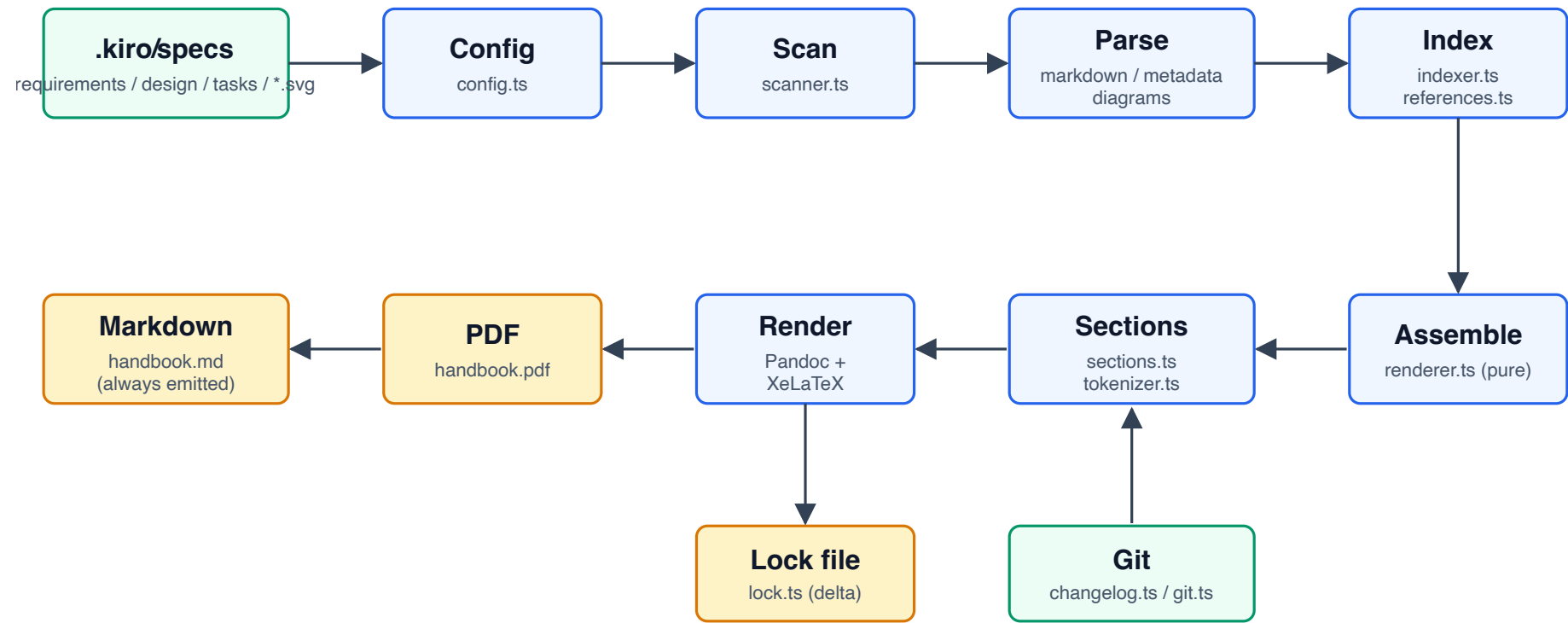
9.1.3 Tasks

- ☒ 1. Configuration loading
 - Load and normalise `handbook.yml`; locate repo and tool roots; apply defaults.
 - *Requirements:* [5.4](#)
- ☒ 2. Spec discovery and parsing
 - ☒ 2.1 Discover spec directories and classify documents
 - ★ *Requirements:* [1.1](#), [1.2](#), [1.3](#), [1.4](#)
 - ☒ 2.2 Sort specs by stable id after discovery
 - ★ *Requirements:* [1.5](#), [2.2](#)
 - ☒ 2.3 Compute deterministic content hashes
 - ★ *Requirements:* [4.3](#)
- ☒ 3. Stable identifiers
 - Derive spec ids from front matter or slug; derive requirement/task/diagram ids.
 - *Requirements:* [2.1](#), [2.2](#), [2.3](#), [2.4](#)
- ☒ 4. Indexes and cross-references
 - ☒ 4.1 Build specification, requirements, tasks, and diagram indexes
 - ★ *Requirements:* [3.1](#)
 - ☒ 4.2 Linkify requirement references from tasks
 - ★ *Requirements:* [3.2](#)
 - ☒ 4.3 Emit the traceability matrix
 - ★ *Requirements:* [3.3](#)
 - ☒ 4.4 Assign handbook-wide figure numbers
 - ★ *Requirements:* [3.4](#)
- ☒ 5. Deterministic assembly
 - Order sections and separate parts with page breaks; resolve the build date.
 - *Requirements:* [4.1](#), [4.2](#)
- ☒ 6. Rendering and graceful degradation
 - ☒ 6.1 Assemble Markdown (pure) and write it to the output directory
 - ★ *Requirements:* [4.1](#), [5.1](#)
 - ☒ 6.2 Detect Pandoc/LaTeX and render the PDF when available

- ✱ *Requirements:* [5.1](#)
- ☒ 6.3 Convert and embed SVG diagrams; reference them when no converter exists
 - ✱ *Requirements:* [5.2](#)
- ☒ 6.4 Render mermaid via mmdc; fall back to fenced code when absent
 - ✱ *Requirements:* [5.3](#)
- ☒ 7. Lock file and delta detection
 - Write `handbook.lock.json` and report added/changed/removed specs.
 - *Requirements:* [4.3](#)

9.1.4 Diagrams

Handbook Generation Pipeline



Optional tooling (Pandoc, SVG converter, mmdc) degrades gracefully — Markdown is always produced.

Figure 1: Pipeline

9.2 PDF Chunking — SPEC-PDF-CHUNKING

Version: 1.0 · **Status:** implemented · **Slug:** pdf-chunking

9.2.1 Requirements

Introduction Kiro limits attachments to 4 MB, but a generated handbook is often larger. PDF chunking splits a large PDF into a sequence of smaller, fully valid PDFs that can be uploaded individually and read in order. Splitting is page-based so every chunk opens on its own.

Requirements

Requirement 1: Page-based splitting into valid PDFs **User Story:** As a user uploading to Kiro, I want a large PDF split into standalone files, so that each one opens and reads correctly on its own.

Acceptance Criteria

1. WHEN a PDF exceeds the target chunk size THEN the system SHALL pack whole pages into each chunk until adding the next page would exceed the target.
2. WHEN a chunk is written THEN the system SHALL produce a fully valid, standalone PDF with correct header, objects, xref table, and trailer.
3. WHEN all pages are distributed THEN the system SHALL preserve the original page order across chunks.
4. WHEN the entire PDF already fits within the target THEN the system SHALL emit a single `_chunk_1_of_1` file.

Requirement 2: Configurable and auto-detected size **User Story:** As a user, I want to choose the chunk size or let the tool pick one, so that I can balance the number of uploads against the size limit.

Acceptance Criteria

1. WHEN a size of 1, 2, 3, or 4 MB is provided THEN the system SHALL use it as the per-chunk target.
2. WHEN an invalid size is provided THEN the system SHALL reject it with a clear error.
3. WHEN auto-size is requested THEN the system SHALL suggest a size that keeps each chunk within Kiro's 4 MB limit.

Requirement 3: Oversized single pages **User Story:** As a user, I want the tool to handle a page that is larger than the target on its own, so that the process does not stall.

Acceptance Criteria

1. WHEN a single page exceeds the target size THEN the system SHALL emit that page as its own chunk and warn that it is oversized.
2. WHEN emitting an oversized page THEN the system SHALL continue processing the remaining pages normally.

Requirement 4: Predictable naming and dry-run **User Story:** As a user, I want predictable file names and a preview mode, so that I can see the plan before writing files.

Acceptance Criteria

1. WHEN chunks are created THEN the system SHALL name them `<name>_chunk_<i>_of_<N>.pdf`.
2. WHEN dry-run is requested THEN the system SHALL report what would be created without writing any files.

Chunking Flow

Figure 2: Chunking Flow

3. WHEN chunking completes THEN the system SHALL report total size, chunk size, chunk count, and per-chunk page counts and sizes.

9.2.2 Design

Overview `pdf-chunker.ts` reads a PDF with `pdf-lib` and greedily groups pages into chunks that each stay at or under a target size. Every chunk is rebuilt as a new `PDFDocument`, so the output is always a valid, standalone PDF.

Algorithm

1. Read the input file and load it with `pdf-lib` (`ignoreEncryption: true`).
2. If the whole file already fits the target, re-save it as `_chunk_1_of_1`.
3. Otherwise, iterate pages, maintaining a current group:
 - Estimate the single-page size as an upper bound.
 - Build a tentative chunk from `current + page` and measure its real serialised size (copied objects like fonts/images can be shared, so the real size is often smaller than the naive sum).
 - If the tentative size overshoots and `current` is non-empty, close the current group and start a new one with this page.
 - If a single page alone exceeds the target, emit it as its own oversized chunk with a warning.
4. Serialise each group to `<name>_chunk_<i>_of_<N>.pdf` (unless dry-run).

Measuring the real serialised size after each addition — rather than summing per-page estimates — is what keeps chunks close to, but under, the target.

Interfaces

- `chunkPdf(options: ChunkOptions): Promise<ChunkResult>` — the splitter.
- `suggestChunkSize(bytes): 1 | 2 | 3 | 4` — auto-size heuristic (prefers 4 MB to minimise uploads).
- `isValidChunkSize(size): boolean` — guards CLI input.

`ChunkResult` reports total size, chosen chunk size, count, and per-chunk path, byte size, human size, and page count.

CLI Exposed as `handbook chunk` and the standalone `pdf-chunker` binary, with `--input`, `--size`, `--output`, `--dry-run`, and `--auto-size` flags. Paths are resolved relative to the repository root.

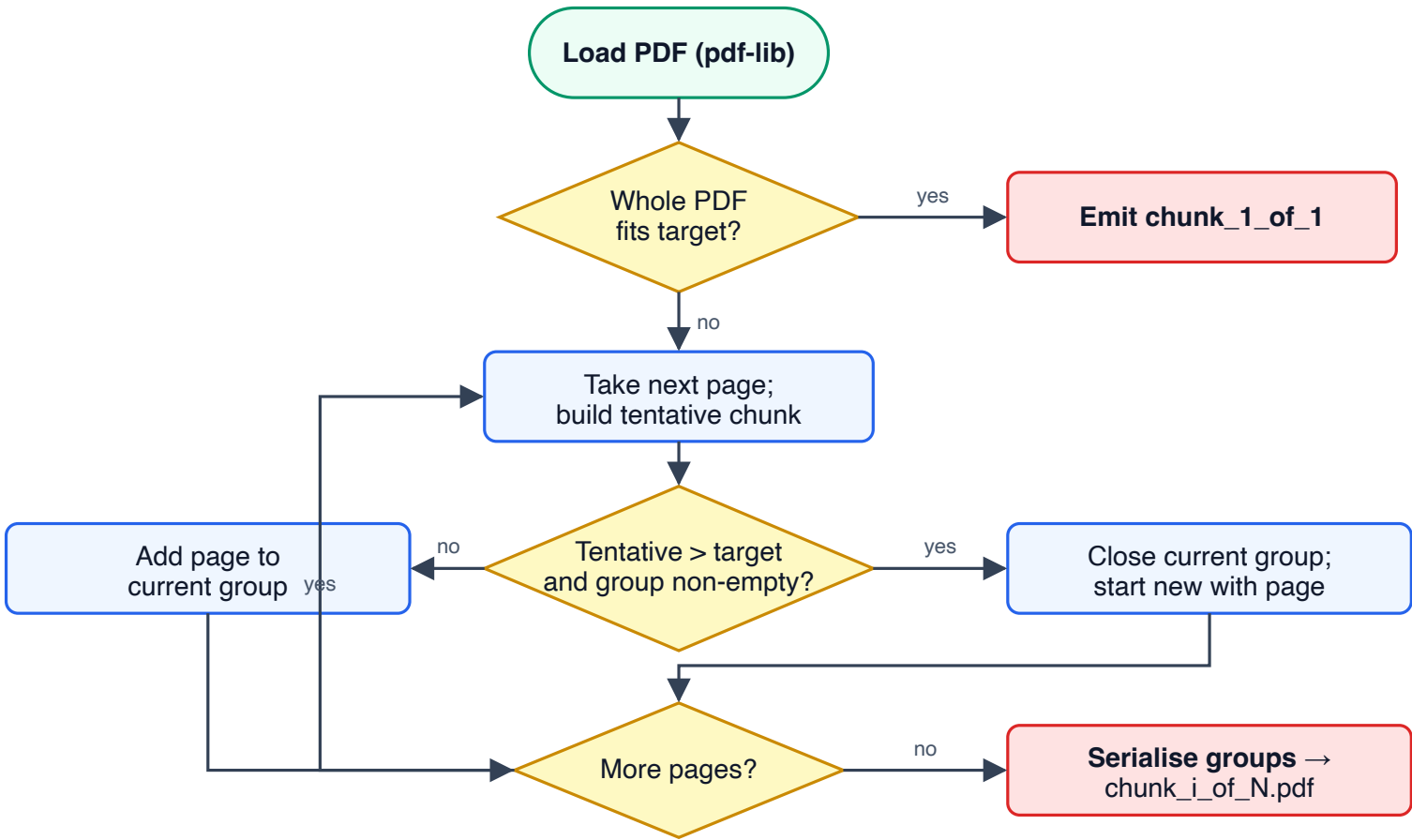
Testing Strategy Tests assert that chunks are valid parseable PDFs, that pages are preserved and ordered across chunks, that sizes stay within target where pages allow, that naming includes the total count, and that dry-run writes nothing.

9.2.3 Tasks

- ☒ 1. Load and measure the input PDF
 - Read bytes and parse with `pdf-lib` (`ignoreEncryption`); report size and page count.
 - *Requirements:* [1.1](#)
- ☒ 2. Single-chunk fast path
 - Re-save as `_chunk_1_of_1` when the whole PDF fits the target.

- *Requirements: 1.4*
- ☒ 3. Greedy page packing
 - ☒ 3.1 Measure real serialised size of each tentative chunk
 - ★ *Requirements: 1.1, 1.2*
 - ☒ 3.2 Close and start groups on overshoot; preserve page order
 - ★ *Requirements: 1.1, 1.3*
 - ☒ 3.3 Emit oversized single pages with a warning
 - ★ *Requirements: 3.1, 3.2*
- ☒ 4. Size selection
 - ☒ 4.1 Validate explicit sizes (1–4 MB)
 - ★ *Requirements: 2.1, 2.2*
 - ☒ 4.2 Auto-size heuristic within the 4 MB limit
 - ★ *Requirements: 2.3*
- ☒ 5. Output, naming, and reporting
 - ☒ 5.1 Name chunks `<name>_chunk_<i>_of_<N>.pdf`
 - ★ *Requirements: 4.1*
 - ☒ 5.2 Dry-run preview without writing files
 - ★ *Requirements: 4.2*
 - ☒ 5.3 Summary of totals and per-chunk stats
 - ★ *Requirements: 4.3*
- ☒ 6. CLI wiring
 - Expose `handbook chunk` and the `pdf-chunker` binary with all flags.
 - *Requirements: 2.1, 4.2*

PDF Chunking — Page Packing



A single page larger than the target is emitted as its own oversized chunk with a warning.

Figure 2: Chunking Flow

9.3 Tokenizer Summary — SPEC-TOKENIZER-SUMMARY

Version: 1.0 · **Status:** implemented · **Slug:** tokenizer-summary

9.3.1 Requirements

Introduction Specs and steering documents are consumed by LLMs with finite context windows. The tokenizer summary appendix estimates token counts for every spec and steering document across configurable models, so authors can spot and trim bloat. Counts are deterministic, offline approximations — good enough to compare documents and track growth over time.

Requirements

Requirement 1: Per-model token estimation **User Story:** As a spec author, I want token estimates for the models I use, so that I can see how my documents fit different context budgets.

Acceptance Criteria

1. WHEN a model is configured THEN the system SHALL estimate tokens using a characters-per-token ratio for that model.
2. WHEN counting THEN the system SHALL normalise line endings and collapse whitespace so counts are identical across platforms.
3. WHEN content is empty THEN the system SHALL report zero tokens; WHEN content is non-empty THEN the estimate SHALL be at least one.

Requirement 2: Configurable models **User Story:** As a user, I want to choose which models appear and tune their ratios, so that the table matches my toolchain.

Acceptance Criteria

1. WHEN model keys are configured THEN the system SHALL render one column per model in the summary tables.
2. WHEN a `chars_per_token` override is provided THEN the system SHALL use it in place of the built-in ratio.
3. WHEN an unknown model key is provided THEN the system SHALL fall back to a generic ratio rather than failing.
4. WHEN no models are configured THEN the system SHALL default to a single model.

Requirement 3: Summary appendix **User Story:** As a reviewer, I want a summary table in the handbook, so that bloat is visible at a glance.

Acceptance Criteria

1. WHEN the tokenizer is enabled THEN the system SHALL emit an “Appendix B — Tokenizer Summary” containing a specifications table and a steering table.
2. WHEN building a table THEN the system SHALL include characters, words, and a token count per model, and a totals row.
3. WHEN ordering rows THEN the system SHALL sort largest-first by the primary model’s token count so the biggest documents surface first.
4. WHEN the tokenizer is disabled THEN the system SHALL emit no summary.

9.3.2 Design

Overview Token estimation lives in `tokenizer.ts` (pure counting + model profiles) and is rendered into the handbook by `tokenSummary()` in `sections.ts`, which is folded into the Appendices as Appendix B.

Why a heuristic, not a real tokenizer Exact tokenization needs model-specific BPE vocabularies (tiktoken for GPT, Anthropic's tokenizer for Claude). These are heavy and network-fetched, which would break the tool's deterministic, offline-friendly builds. Instead we use the well-established characters-per-token heuristic with per-model ratios that can be overridden in config. The result is stable and dependency-free — accurate enough to compare documents and track bloat, and clearly labelled as an estimate.

Model profiles `TokenizerProfile { key, label, charsPerToken }`. Built-ins:

- Claude family (opus-4.8, sonnet-4.5, haiku) $\approx$ 3.5 chars/token.
- GPT family (gpt-4o, gpt-4, gpt-3.5-turbo) $\approx$ 4.0 chars/token.

`resolveProfile(key, overrides)` applies a `chars_per_token` override and falls back to a generic $\sim$ 4.0 profile for unknown keys. `resolveProfiles([])` defaults to opus-4.8.

Counting `countTokens(text, profile)` normalises `\r\n` $\rightarrow$ `\n`, collapses whitespace runs, then divides the character length by the profile ratio (min 1 for non-empty content). `countChars` and `countWords` support the table's other columns.

Rendering `tokenSummary(handbook, config)`:

1. Returns '' when `config.tokenizer.enabled` is false.
2. Resolves the configured profiles.
3. Builds a row per spec (all documents concatenated) and per steering document (front matter stripped).
4. Emits two tables (Specifications, Steering Documents), each sorted largest-first by the primary model, with a bold totals row.

`appendices(handbook, extra[])` accepts the rendered summary and appends it after Appendix A so it nests cleanly under the Appendices heading.

Testing Strategy Unit tests cover platform-stable counting, the Claude-vs-GPT density difference, profile resolution and overrides, the empty/disabled cases, and that the summary renders under Appendices as Appendix B.

9.3.3 Tasks

- ☒ 1. Token counting primitives
 - Implement `countChars`, `countWords`, `countTokens` with platform-stable normalisation.
 - *Requirements:* [1.1](#), [1.2](#), [1.3](#)
- ☒ 2. Model profiles and resolution
 - ☒ 2.1 Define built-in Claude and GPT profiles
 - ★ *Requirements:* [1.1](#), [2.1](#)
 - ☒ 2.2 Apply per-model `chars_per_token` overrides
 - ★ *Requirements:* [2.2](#)
 - ☒ 2.3 Fall back to a generic profile for unknown keys
 - ★ *Requirements:* [2.3](#)
 - ☒ 2.4 Default to a single model when none configured
 - ★ *Requirements:* [2.4](#)
- ☒ 3. Summary appendix rendering
 - ☒ 3.1 Build per-spec and per-steering rows
 - ★ *Requirements:* [3.1](#), [3.2](#)
 - ☒ 3.2 Sort largest-first and add a totals row
 - ★ *Requirements:* [3.2](#), [3.3](#)
 - ☒ 3.3 Fold the summary into Appendices as Appendix B

- ✱ *Requirements:* [3.1](#)
- ☒ 3.4 Emit nothing when disabled
 - ✱ *Requirements:* [3.4](#)
- ☒ 4. Configuration
 - Add the `tokenizer` block to config types and defaults.
 - *Requirements:* [2.1](#), [2.2](#), [3.4](#)

10 Architecture Decisions

No architecture decision records found in the source tree.

11 Traceability Matrix

Requirement → Design → Task → Code/Test, derived from spec cross references.

Requirement ID	Requirement	Design	Tasks	Code/Test
SPEC-HANDBOOK-GENERATION:R1	Req 1	design	2.1, 2.2	see tasks
SPEC-HANDBOOK-GENERATION:R2	Req 2	design	2.2, 3	see tasks
SPEC-HANDBOOK-GENERATION:R3	Req 3	design	4.1, 4.2, 4.3, 4.4	see tasks
SPEC-HANDBOOK-GENERATION:R4	Req 4	design	2.3, 5, 6.1, 7	see tasks
SPEC-HANDBOOK-GENERATION:R5	Req 5	design	1, 6.1, 6.2, 6.3, 6.4	see tasks
SPEC-PDF-CHUNKING:R1	Req 1	design	1, 2, 3.1, 3.2	see tasks
SPEC-PDF-CHUNKING:R2	Req 2	design	4.1, 4.2, 6	see tasks
SPEC-PDF-CHUNKING:R3	Req 3	design	3.3	see tasks
SPEC-PDF-CHUNKING:R4	Req 4	design	5.1, 5.2, 5.3, 6	see tasks
SPEC-TOKENIZER-SUMMARY:R1	Req 1	design	1, 2.1	see tasks
SPEC-TOKENIZER-SUMMARY:R2	Req 2	design	2.1, 2.2, 2.3, 2.4, 4	see tasks
SPEC-TOKENIZER-SUMMARY:R3	Req 3	design	3.1, 3.2, 3.3, 3.4, 4	see tasks

12 Git Change History

No git history available.

13 Appendices

13.1 Appendix A – Reference Files

No supplementary reference files found.

13.2 Appendix B – Tokenizer Summary

Estimated token counts for each specification and steering document, to help identify bloat. Counts are deterministic approximations based on characters per token (Claude Opus 4.8 $\approx$ 3.5 chars/token; Claude Haiku $\approx$ 3.5 chars/token; GPT-4o (o200k) $\approx$ 4 chars/token). They are estimates, not exact tokenizer output.

13.2.1 Specifications

Spec ID	Title	Chars	Words	Claude Opus 4.8	Claude Haiku	GPT-4o (o200k)
SPEC-HANDBOOK-GENERATION	Handbook Generation	8,415	1,217	2,341	2,341	2,049
SPEC-PDF-CHUNKING	PDF Chunking	5,868	947	1,628	1,628	1,425
SPEC-TOKENIZER-SUMMARY	Tokenizer Summary	5,464	814	1,525	1,525	1,334
Total		19,747	2,978	5,494	5,494	4,808

13.2.2 Steering Documents

Document	Chars	Words	Claude Opus 4.8	Claude Haiku	GPT-4o (o200k)
structure.md	2,616	347	710	710	621
tech.md	1,942	287	539	539	472
diagramming.md	1,843	284	519	519	454
product.md	1,458	231	411	411	360
Total	7,859	1,149	2,179	2,179	1,907